AI ASSISTED CODING
ASSIGNMENT - 18.4
2303A510F2 - Anitha
Batch - 14

Lab 18: API Integration: Connecting to External Services with Error Handling

Lab Objectives

• Learn how to integrate Python programs with external REST APIs.

• Understand API request/response handling using requests or similar libraries.

• Implement proper error handling for failed API calls (timeouts, invalid responses, rate limits).

• Practice extracting and displaying meaningful information from API responses.

Learning Outcome

After completing this assignment, students will be able to:

• Integrate external REST APIs using AI-assisted Python scripting.

• Implement structured error handling for API failures and invalid inputs.

• Safely parse and validate JSON responses from external services.

• Develop resilient applications capable of handling real-world API limitations.

Task 1 – Movie Database API Integration

Scenario

You are building a movie recommendation tool that fetches movie details from a public Movie Database API.

Task

Use AI to generate a Python script that connects to a Movie API and retrieves movie information by title.

Instructions

- Use requests library to fetch movie details.
- Accept movie title as user input.
- Display:
    - o Title
    - o Release year
    - o Rating
    - o Plot summary
- Implement error handling for:
    - o Invalid movie name
    - o API key errors
    - o Network timeout
    - o Empty API response

Expected Output

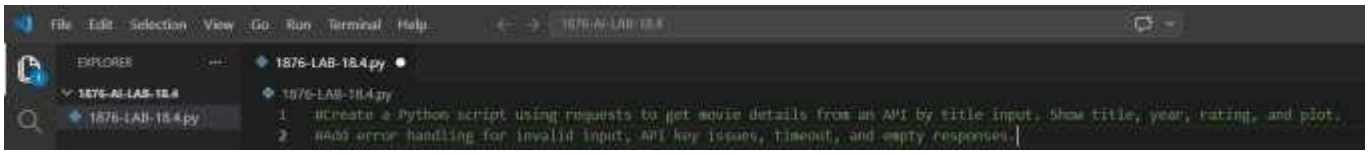
- A working Python script that fetches movie details.
- Graceful error messages instead of crashes.
- Clean, readable output format.

Movie Database API Integration

Prompt

#Create a Python script using requests to get movie details from an API by title input. Show title, year, rating, and plot. Add error handling for invalid input, API key issues, timeout, and empty responses.

Prompt Screenshot



Code

```python
#Create a Python script using requests to get movie details from an API by title input. Show title, year, rating, and plot.
# Add error handling for invalid input, API key issues, timeout, and empty responses.
import requests
API_KEY = "eb0d2834"
BASE_URL = "http://www.omdbapi.com/"
def get_movie_details(title):
    try:
        response = requests.get(
            BASE_URL,
            params={"t": title, "apikey": API_KEY},
            timeout=5
        )
        if response.status_code == 401:
            print("Error: Unauthorized. Check your API key.")
            return
        response.raise_for_status()
        data = response.json()
        if not data:
            print("Error: Empty response from API.")
            return
        if data.get("Response") == "False":
            print(f"Error: {data.get('Error')}")
            return
        title = data.get("Title", "N/A")
        year = data.get("Year", "N/A")
        rating = data.get("imdbRating", "N/A")
        plot = data.get("Plot", "N/A")
        print("\n🎬 Movie Details")
        print("----------------------------")
        print(f"Title       : {title}")
        print(f"Year        : {year}")
        print(f"IMDb Rating : {rating}")
        print(f"Plot        : {plot}")
    except requests.exceptions.Timeout:
        print("Error: Request timed out. Please try again.")
    except requests.exceptions.ConnectionError:
        print("Error: Network problem. Check your internet connection.")
    except requests.exceptions.HTTPError as e:
        print(f"HTTP Error occurred: {e}")
    except ValueError:
        print("Error: Invalid JSON response.")
    except Exception as e:
        print(f"Unexpected error: {e}")
if __name__ == "__main__":
    movie_title = input("Enter movie title: ").strip()
    if not movie_title:
        print("Error: Movie title cannot be empty.")
    else:
        get_movie_details(movie_title)
```

Output

```
PS C:\Users\91986\OneDrive\Documents\1876-AI-LAB-18.4> & "C:/Program Files/Python313/python.exe" c:/Users/91986/OneDrive/Documents/1876-AI-LAB-18.4/1876-LAB-18.4.py
Enter movie title: Inception

🎬 Movie Details
----------------------------
Title       : Inception
Year        : 2010
IMDb Rating : 8.8
Plot        : A thief who steals corporate secrets through the use of dream-sharing technology is given the inverse task of planting an idea into the mind of a CEO, but his tragic past may doom the pro
ject and his team to disaster.
```

Task 2 – Public Transport Schedule API

Scenario

A city transport app needs to fetch bus/train arrival timings from a transport API.

Task

Use AI assistance to integrate a transport schedule API.

Instructions

• Fetch arrival times for a given station/stop ID.

• Validate user input.

• Handle errors such as:

o Invalid stop ID

o Server unavailable

o Timeout errors

• Display next 3 arrival times in formatted output.

• Implement retry logic (minimum 1 retry

attempt). Expected Output

• Python script showing upcoming arrivals.

• Retry mechanism working properly.

• Proper exception handling using try-except blocks.

Public Transport Schedule API

Prompt

#Write a Python program using requests to fetch public transport arrival times from an API. Take stop ID as input and display the next 3 arrivals.Add error handling for invalid stop ID, server errors, and timeout, and include retry logic.

Prompt Screenshot

Code

```python
#Write a Python program using requests to fetch public transport arrival times from an API. Take stop ID as input and display the next 3 arrivals.
#Add error handling for invalid stop ID, server errors, and timeout, and include retry logic.
import requests
import time
BASE_URL = "https://api-v3.mbta.com/predictions"
def get_arrival_times(stop_id, retries=1):
    attempt = 0
    while attempt <= retries:
        try:
            response = requests.get(
                BASE_URL,
                params={"filter[stop]": stop_id},
                timeout=5
            )
            if response.status_code >= 500:
                print("Server error. Retrying...")
                attempt += 1
                time.sleep(2)
                continue
            response.raise_for_status()
            data = response.json()
            arrivals = data.get("data", [])
            if not arrivals:
                print("Error: Invalid stop ID or no arrival data found.")
                return
            print("\n Next 3 Arrival Times")
            print("----------------------------")
            count = 0
            for item in arrivals:
                if count >= 3:
                    break
                arrival_time = item["attributes"].get("arrival_time")
                if arrival_time:
                    print(f"{count + 1}. {arrival_time}")
                    count += 1
            if count == 0:
                print("No upcoming arrivals available.")
            return
        except requests.exceptions.Timeout:
            print("Timeout occurred. Retrying...")
            attempt += 1
        except requests.exceptions.ConnectionError:
            print("Network error. Please check your connection.")
            return
        except requests.exceptions.HTTPError as e:
            print(f"HTTP Error: {e}")
            return
        except ValueError:
            print("Error: Invalid JSON response.")
            return
        except Exception as e:
            print(f"Unexpected error: {e}")
            return
    print("Failed after retrying. Please try again later.")
if __name__ == "__main__":
    stop_id = input("Enter stop ID: ").strip()
    if not stop_id:
        print("Error: Stop ID cannot be empty.")
    else:
        get_arrival_times(stop_id)
```

Output

```
PS C:\Users\91986\OneDrive\Documents\1876-AI-LAB-18.4> & "C:/Program Files/Python313/python.exe" c:/Users/91986/OneDrive/Documents/1876-AI-LAB-18.4/1876-LAB-18.4.py
Enter stop ID: place-sstat

 Next 3 Arrival Times
----------------------------
1. 2026-03-26T04:17:19-04:00
2. 2026-03-26T04:17:19-04:00
3. 2026-03-26T04:17:19-04:00
```

Observation

- Program takes stop ID as input and fetches arrival times.
- Displays next 3 arrivals when data is available.

- Shows error message for invalid stop ID.
- Retry works during timeout or server issues.

Explanation

- Uses requests to call transport API.
- Parses JSON response to get arrival times.
- Uses try-except for handling errors.
- Retry logic handles temporary failures.

Justification

- API provides real-time data.
- Error handling prevents crashes.
- Retry improves reliability.
- Limited output (3 results) keeps it clear.

Conclusion

- Successfully integrates API with Python.
- Handles real-world errors effectively.
- Program is reliable and user-friendly.

Task 3 – Stock Market Data API

Scenario

You are developing a finance dashboard that retrieves live stock prices.

Task

Use AI to generate a Python script that fetches stock data from a financial API.

Instructions

• Accept stock symbol as input.

• Retrieve:

    o Current price

    o Day high

    o Day low

- Handle:
    - o Invalid stock symbol
    - o API rate limits
    - o Missing fields in JSON response
- Validate and safely parse JSON response.

Expected Output

- Script fetches and displays stock information.
- No unhandled exceptions.
- Clear error messages for invalid symbols or rate limits.

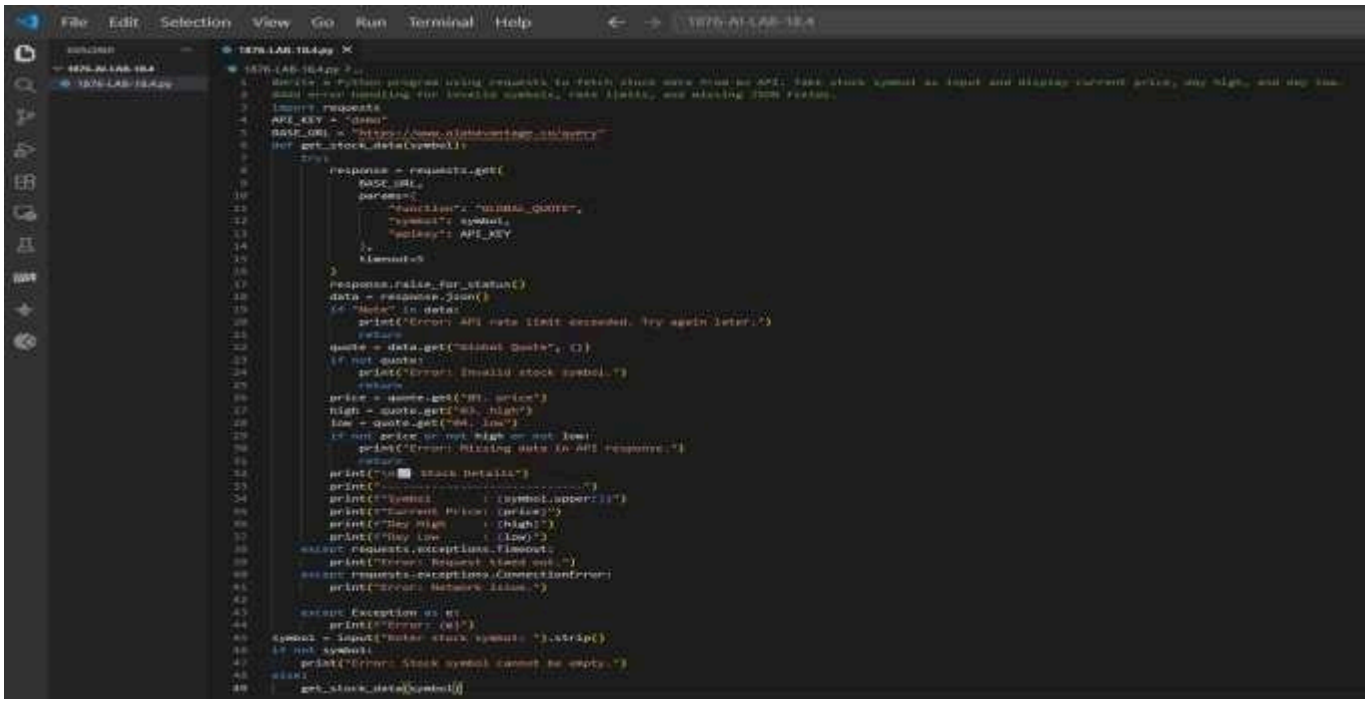
Stock Market Data API

Prompt

#Write a Python program using requests to fetch stock data from an API. Take stock symbol as input and display current price, day high, and day low.Add error handling for invalid symbols, rate limits, and missing JSON fields.

Prompt Screenshot

Code



Output

```
PROBLEMS 10   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE                                   Python
PS C:\Users\91986\OneDrive\Documents\1876-AI-LAB-18.4> & "C:/Program Files/Python313/python.exe" c:/Users/91986/OneDrive/Documents/1876-AI-LAB-18.4/1876-LAB-18.4.py
Enter stock symbol: IBM

📊 Stock Details
----------------------------
Symbol       : IBM
Current Price: 241.3900
Day High     : 246.1900
Day Low      : 238.0000
```

Observation

- Program accepts stock symbol and fetches data.
- Displays current price, high, and low.
- Shows error for invalid symbol or missing data.
- Handles API rate limit errors properly.

Explanation

- Uses requests to call stock API.
- Parses JSON response safely.
- Extracts required stock values.
- Uses try-except for error handling.

Justification

- API provides real-time financial data.
- Error handling prevents crashes.
- Validation ensures correct output.
- Safe parsing avoids missing field issues.

Conclusion

- Successfully integrates stock API with Python.
- Handles errors like invalid symbols and rate limits.
- Program is reliable and user-friendly.

Task 4 – Geolocation API (IP Address Lookup)

Scenario

A cybersecurity tool needs to detect the geographical location of a user based on IP address.

Task

Integrate a Geolocation API using AI assistance.

Instructions

- Accept IP address as input.
- Fetch:
    - o Country
    - o City
    - o ISP
- Validate IP format before making API call.
- Handle:
    - o Invalid IP format
    - o API failure
    - o JSON decoding errors

Expected Output

- A script that returns geolocation details.
- Input validation implemented.
- Robust error handling included.

Geolocation API (IP Address Lookup)

Prompt

#Write a Python program to fetch geolocation details of an IP address using an API. Validate IP format and display country, city, and ISP. Include error handling for invalid IP, API failures, and JSON errors.

Prompt Screenshot

```
#write a Python program to fetch geolocation details of an IP address using an API.
#Validate IP format and display country, city, and ISP. Include error handling for invalid IP, API failures, and JSON errors.
```

Code

```python
#write a Python program to fetch geolocation details of an IP address using an API.
#Validate IP format and display country, city, and ISP. Include error handling for invalid IP, API failures, and JSON errors.
import requests
import re
import time
BASE_URL = "http://ip-api.com/json/"
def is_valid_ip(ip):
    """Validate IPv4 format"""
    pattern = r"^\d{1,3}(\.\d{1,3}){3}$"
    if not re.match(pattern, ip):
        return False
    octets = ip.split(".")
    return all(0 <= int(o) <= 255 for o in octets)
def get_geolocation(ip, retries=2):
    """Fetch geolocation details with retry logic"""
    if not is_valid_ip(ip):
        print("Error: Invalid IP format.")
        return
    attempt = 0
    while attempt <= retries:
        try:
            response = requests.get(BASE_URL + ip, timeout=15)
            response.raise_for_status()
            data = response.json()
            if data.get("status") != "success":
                print(f"Error: {data.get('message', 'API failed')}")
                return
            country = data.get("country", "N/A")
            city = data.get("city", "N/A")
            isp = data.get("isp", "N/A")
            print("\n🌍 Geolocation Details")
            print("----------------------------")
            print(f"IP Address : {ip}")
            print(f"Country    : {country}")
            print(f"City       : {city}")
            print(f"ISP        : {isp}")
            return
        except requests.exceptions.Timeout:
            attempt += 1
            print(f"Timeout occurred. Retrying... ({attempt}/{retries})")
            time.sleep(2)
        except requests.exceptions.ConnectionError:
            print("Error: Network issue.")
            return
        except ValueError:
            print("Error: Failed to parse JSON response.")
            return
        except Exception as e:
            print(f"Error: {e}")
            return
    print("Failed after multiple attempts. Please try again later.")
ip_address = input("Enter IP address: ").strip()
get_geolocation(ip_address)
```

Output

```
PS C:\Users\91986\OneDrive\Documents\1876-AI-LAB-18.4> & "C:/Program Files/Python313/python.exe" c:/Users/91986/OneDrive/Documents/1876-AI-LAB-18.4/1876-LAB-18.4.py
Enter IP address: 8.8.8.8

🌍 Geolocation Details
----------------------------
IP Address : 8.8.8.8
Country    : United States
City       : Ashburn
ISP        : Google LLC
```

Observation

- Program accepts IP input and fetches geolocation.
- Validates IP format before API call.
- Displays country, city, and ISP.
- Shows error for invalid IP, API failure, or network issues.

Explanation

- Uses requests to call the geolocation API.
- Checks IP format using a regex.
- Parses JSON response safely.
- Uses try-except to handle timeouts, connection errors, and invalid JSON.

Justification

- Input validation prevents unnecessary API calls.
- Error handling ensures program does not crash.
- Safe JSON parsing avoids missing fields issues.
- Provides clear, user-friendly output.

Conclusion

- Successfully integrates Geolocation API with Python.
- Handles invalid IPs and API errors effectively.
- Program is robust, reliable, and user-friendly

Task 5 – Real-Time Application: Payment Gateway Simulation API

Scenario

You are building a test payment integration system that connects to a sandbox payment API.

Task

Use AI to generate a Python script that simulates payment processing.

Instructions

- Send POST request with:
    - o Amount
    - o Currency
    - o Payment method
- Validate input values.
- Handle:
    - o Invalid payment details
    - o HTTP status errors (400, 401, 500)
    - o Network interruptions
- Display transaction status clearly.
- Log failed transactions to a file.

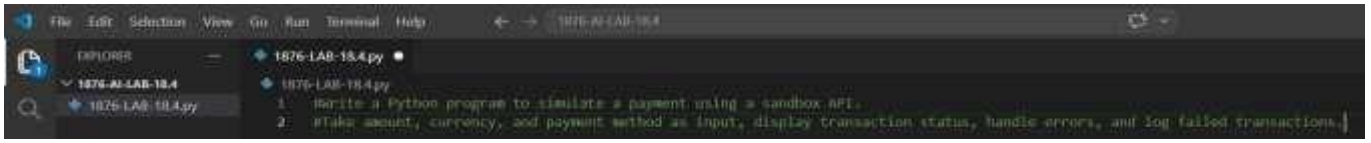
Expected Output

- Script processes simulated payment.
- Errors handled gracefully.
- Failed requests logged properly.

Prompt

#Write a Python program to simulate a payment using a sandbox API. Take amount, currency, and payment method as input, display transaction status, handle errors, and log failed transactions.

Prompt Screenshot



Code

```python
#Write a Python program to simulate a payment using a sandbox API.
#Take amount, currency, and payment method as input, display transaction status, handle errors, and log failed transactions.
import requests
import json
import time
API_URL = "https://sandbox.payments.example.com/process"  # Placeholder sandbox API URL
LOG_FILE = "failed_transactions.log"
def validate_input(amount, currency, method):
    """Validate payment inputs"""
    if amount <= 0:
        return False, "Amount must be greater than 0."
    if not currency.isalpha() or len(currency) != 3:
        return False, "Currency must be a 3-letter code."
    if method.lower() not in ["card", "paypal", "bank"]:
        return False, "Payment method must be card, paypal, or bank."
    return True, ""
def log_failure(data, error):
    """Append failed transactions to log file"""
    with open(LOG_FILE, "a") as f:
        f.write(f"{time.asctime()} - Failed transaction: {data} - Error: {error}\n")
def process_payment(amount, currency, method):
    payload = {
        "amount": amount,
        "currency": currency.upper(),
        "payment_method": method.lower()
    }
    try:
        response = requests.post(API_URL, json=payload, timeout=10)
        if response.status_code >= 500:
            print(f"Server error ({response.status_code}). Try again later.")
            log_failure(payload, f"Server error {response.status_code}")
            return
        elif response.status_code == 400:
            print("Error: Invalid payment details.")
            log_failure(payload, "HTTP 400 - Bad Request")
            return
        elif response.status_code == 401:
            print("Error: Unauthorized. Check API credentials.")
            log_failure(payload, "HTTP 401 - Unauthorized")
            return
        data = response.json()
        status = data.get("status")
        message = data.get("message", "")
        if status == "success":
            print("\n✅ Payment Successful")
            print(f"Amount   : {amount} {currency.upper()}")
            print(f"Method   : {method}")
            print(f"Message  : {message}")
```


```python
def process_payment(amount, currency, method):
        if status == "success":
            print("\n✅ Payment Successful")
            print(f"Amount   : {amount} {currency.upper()}")
            print(f"Method   : {method}")
            print(f"Message  : {message}")
```


```python
        else:
            print("\n❌ Payment Failed")
            print(f"Reason: {message}")
            log_failure(payload, message)
    except requests.exceptions.Timeout:
        print("Error: Request timed out.")
        log_failure(payload, "Timeout")

    except requests.exceptions.ConnectionError:
        print("Error: Network issue.")
        log_failure(payload, "ConnectionError")

    except ValueError:
        print("Error: Failed to parse API response.")
        log_failure(payload, "Invalid JSON")

    except Exception as e:
        print(f"Unexpected error: {e}")
        log_failure(payload, str(e))
try:
    amount = float(input("Enter amount: ").strip())
    currency = input("Enter currency (3-letter code): ").strip()
    method = input("Enter payment method (card/paypal/bank): ").strip()
    valid, error_msg = validate_input(amount, currency, method)
    if not valid:
        print(f"Input Error: {error_msg}")
    else:
        process_payment(amount, currency, method)
except ValueError:
    print("Error: Invalid input for amount.")
```

Output

```
PROBLEMS 10   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE                                   Python
PS C:\Users\91986\OneDrive\Documents\1876-AI-LAB-18.4> & "C:/Program Files/Python313/python.exe" c:/Users/91986/OneDrive/Documents/1876-AI-LAB-18.4/1876-LAB-18.4.py
Enter amount: 100
Enter currency (3-letter code): USD
Enter payment method (card/paypal/bank): card
Error: Network Issue.
```

Observation

- Program accepts amount, currency, and payment method.
- Sends POST request to sandbox API.
- Displays transaction status clearly.
- Logs failed transactions in a file.
- Handles HTTP errors, invalid input, network issues, and JSON parsing errors.

Explanation

- requests.post used for API call.
- Input validated to prevent invalid submissions.
- Errors handled with try-except blocks.
- Failed transactions logged for auditing.

Justification

- Simulates real payment processing safely.
- Logging ensures failed transactions can be reviewed.
- Proper error handling prevents crashes.
- Input validation prevents invalid requests to API.

Conclusion

- Successfully simulates a payment gateway integration.
- Robust against invalid input, network errors, and API failures.
- Provides clear feedback and logs for auditing.